STRUCTURED AGILITY

Requirements Definition Template

A Systematic Approach to Agile Requirements Definition

Version 0.01 · Lionel Lam

A Companion to *A High-level Guide to Structured Agility*.

© 2026 Lionel Lam. Licensed under CC BY 4.0. See the final page for licence and attribution details.

Table of Contents

Table of Contents *2*

Requirements Definition Template *3*

Worked Example *5*

Annex: Design Rationale *7*

A1. Three sections and the stability gradient 7

A2. Progressive elaboration, size and the Definition of Ready (DoR) 7

A3. Work item header kept separate 8

A4. Summary: [Epic] > [Feature] > [Verb] [noun] 8

A5. User story: As a / I want to / So that 9

A6. Design: link to the source 9

A7. Acceptance criteria: two complementary formats 9

A8. Given-When-Then conventions 9

Licence and attribution *10*

Requirements Definition Template

Copy into the description of a new work item (a ticket, in Jira, or the equivalent in your team's tool) and replace the *grey prompts*. You need not complete every section at once: start with Context & Background and add the rest as the work is refined.

In the Summary field, keep epic and feature names short and spelled consistently. Tailor the template appropriately to your team's context and project needs.

Work Item Header *Jira fields*

- **Summary:** *[Epic] > [Feature] > [Verb] [noun], e.g. "Reporting > Monthly usage > Download report"*
- **Epic:** *The feature group this belongs to (the "where" in the product)*
- **Labels:** *Cross-cutting concerns, e.g. Security, Ops, Risks etc.*
- **Size:** *Small enough to start work, meaningful enough to deliver value. Treat poorly defined work items as potentially larger/complex until refined; if the criteria grow too many, split.*

Context & Background *Business*

- **Who is this for:** *The user or role affected*
- **Need / problem:** *What they need, or what is not working today*
- **Why now:** *Business value, priority or deadline*
- **References:** *Links to requests, policies, related work items, supporting documentation etc.*

User Story *Business*

- **As a** *[user / role],*
- **I want to** *[do something],*
- **So that** *[I can achieve a business outcome].*

Design *Design / Experience / Usability Considerations*

- **Intended experience:** *What the user should be able to do, and how it should feel*
- **Design links:** *Link to the specific Figma (or equivalent) frame or prototype*
- **Flow:** *User flow or states, or a link to a flow diagram*
- **Design notes:** *Constraints, edge cases, content or accessibility notes*

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

Acceptance Criteria *Implementation*

Done-When *(quick to write; may be enough where the team shares a clear understanding of work to be done)*

#	What	Done-When
1	*Short label for the outcome*	*The condition(s) under which this is accepted as complete [link to Figma element]*
2		
3		

Given-When-Then *(adds precision where there is complexity, e.g. several roles, states or exceptions)*

#	Scenario	Given	When	Then
1	*Short name*	*[Role] + [current state or precondition]*	*[Role] does one action [link to Figma element]*	*Observable outcome: state change, message, record [link to Figma state]*
2				
3				

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

Worked Example

A fully elaborated work item (fictional, trimmed for illustration). The two acceptance criteria formats complement each other; choose based on the return on time invested.

Done-When is quick to write and may be enough where the team shares a clear understanding of what needs to be done.

Given-When-Then (Gherkin notation) adds precision where there is complexity, as with the states and exceptions here.

Work Item Header *Jira fields*

- **Summary:** Review Workflow > Submission > Submit article for approval
- **Epic:** Review Workflow
- **Size:** Medium

Context & Background *Business*

- **Who is this for:** Content Creators who write articles, and Approvers who review them.
- **Need / problem:** Articles are approved over email, with no clear status or record of decisions.
- **Why now:** Decisions must be traceable, and creators need to see where articles stand.

User Story *Business*

- **As a** Content Creator,
- **I want to** submit an article for an Approver to review,
- **So that** only approved articles are published, with a record of each decision.

Design *Design / Experience / Usability Considerations*

- **Intended experience:** A Content Creator submits an article; an Approver approves or rejects it from a queue.
- **Design links:** Figma: Review Workflow › Submit & Approve (prototype)
- **Flow:** DRAFT → IN_REVIEW → PUBLISHED or REJECTED.

Acceptance Criteria *Implementation*

Done-When

#	What	Done-When
1	**Create and submit**	A Content Creator can create, save and submit an article [Article editor]
2	**Review queue**	Approvers can see submitted articles in a queue and approve or reject them [Review queue]
3	**Decision history**	Every decision is recorded in the article's history, with who made it and when [History panel]

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

Given-When-Then

#	Scenario	Given	When	Then
1	**Block submit when invalid**	An article in DRAFT has missing or invalid required fields [Draft with errors]	The Content Creator clicks Submit [Submit button]	Submission is blocked, errors are highlighted, and the status stays DRAFT [Field error state]
2	**Submit for review**	An article in DRAFT has all required fields valid	The Content Creator clicks Submit [Submit button]	Status becomes IN_REVIEW and the Content Creator can no longer edit it [Locked view]
3	**Approve and publish**	An article is IN_REVIEW	The Approver clicks Approve [Approve button]	Status becomes PUBLISHED and the article is visible per visibility rules
4	**Reject with rationale**	An article is IN_REVIEW	The Approver clicks Reject and gives a reason [Reject dialog]	Status becomes REJECTED, the reason is stored, and the Content Creator is notified [Notification]

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

Annex: Design Rationale

A note on these practices. The methods in this guide draw on established industry practices, combined with practices the author has developed through experience building and delivering projects and products. They are a starting point, not a prescription. Teams should *tailor* the template to their own context and project needs and keep *inspecting and adapting* it as they learn what works.

Tool-agnostic. The examples refer to Jira, the tool the author has used most. The concepts are intended to be universal: adapt field names and conventions to whichever tool your team is most familiar with.

A1. Three sections and the stability gradient

Every work item follows the same layout: Context & Background, then Design, then Acceptance Criteria. Each section has an owner (business, design, implementation) and sits at a different point on a stability gradient: business context (i.e. the "what" and "why") tends to be relatively stable, while implementation detail (i.e. the "how") can be expected to be more fluid. The same layout on every work item lowers cognitive load and builds a shared mental model of what a good work item looks like.

The stability gradient is also intended to encourage more fluid planning, in the spirit of "just-enough requirements": detail that is good enough for the situation at hand. For example, Context & Background may be enough to understand a work item's priority, but not enough to know what to implement. Once its priority has more or less firmed up, the team can focus on refining the design and acceptance criteria. This complements Scrum and its backlog refinement practices, where items nearer the top of the backlog are refined in more detail than those further down.

A2. Progressive elaboration, size and the Definition of Ready (DoR)

The template does not need to be completed top to bottom in one go. A work item can start with just Context & Background (and the User Story) when the business reason is clear enough, with Design and Acceptance Criteria added as the work is refined.

How refinement relates to size. A poorly defined work item should be treated as potentially larger, because unknowns tend to hide work. "Larger" here is relative, in the spirit of relative estimation, and closer in meaning to "more complex". Refinement does not make a work item smaller; it reveals its true size, so the team can decide whether it is right-sized or should be split.

A work item is "ready" (i.e. meets the Definition of Ready) when it is clear enough for work to commence, and small enough to complete within an iteration.

One suggested way to think about this (not a prescriptive rule):

Situation	What it means	What to do
Not yet refined	Size is uncertain; treat it as potentially larger (more complex).	Refine before committing it to a sprint, to meet Definition of Ready (DoR).
Refined, manageable scope	Clear and right-sized.	Ready to start.

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

Situation	What it means	What to do
Refined, too many criteria	Clearly too big to finish comfortably.	Split along Done-When lines or Given-When-Then scenarios.

Done-When lines and Given-When-Then rows make natural split lines, because each describes a distinct, testable piece of behaviour (i.e. Behavior- and Test-driven Development as an organizing principle).

Smaller, similarly sized work items move through the team's workflow more smoothly. Large work items tend to cause queues and bottlenecks: at the start, because they take longer to understand and agree on; and at the end, because they reach testing and review in big batches, often late in a sprint. Small work items keep work arriving at each step steadily, so there are less piles-up, and problems and risks surface earlier.

A3. Work item header kept separate

Summary, *Epic*, *Label* and *Size* map to the tool's own fields (Jira, in these examples) rather than the work item description. Keeping them apart reflects where they live, and these fields drive boards, filters, search and reporting.

A4. Summary: [Epic] > [Feature] > [Verb] [noun]

Hierarchical decomposition. The prefix shows where a work item sits in the product, giving the team structure and situational awareness. Unlike the *Epic* field, the *Summary* field goes nearly everywhere the work item does: the sprint and product backlogs, sprint reports, notifications, emails, exports and pasted links.

Searchability. Consistent epic and feature names seed Jira (or an equivalent tool) with the keywords that power search and filters, so related work items are easy to find.

Verb-noun ending. The final element names what the user will be able to do once the work item is delivered, e.g. "Submit article for approval". Written from the perspective of the user it serves, it echoes the "I want to…" of the User Story, keeping work items focused on features and outcomes rather than internal tasks. For technical work items with no direct user, the same pattern still works from the system's perspective.

Backlog legibility. In the backlog lists and on Kanban boards, every row, or card starts with the same Epic > Feature path, so related items stand out even when ranked far apart. This makes it easier to keep related work together, spot dependencies, balance work across epics, and see when too many features are in progress at once. It is a visual cue: it does not replace human judgement in ordering the backlog, which should still follow value and priority, but it sets the stage for better legibility.

Trade-offs. The epic name now lives in two places, so it can drift if a work item moves or an epic is renamed; update the prefix when that happens. Long prefixes can be truncated on board cards, so keep epic and feature names short. The search benefit depends on an agreed, consistently spelled vocabulary.

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

A5. User story: As a / I want to / So that

The story states the need in the user's voice, alongside the business view in Context & Background. "As a" names the role, "I want to" states the intent, and "So that" keeps the value visible, so the team can judge trade-offs against the outcome rather than the feature alone.

A6. Design: link to the source

Link to the specific Figma (or equivalent) frame or prototype rather than pasting screenshots. The design tool remains as the single source of truth, so the work item does not drift out of date when the design changes, and a link to the exact frame saves the reader from hunting around a large design canvas.

Link acceptance criteria to design elements. In either acceptance criteria table, add links to the specific Figma component or state involved, in whichever row or column helps most: a *Given* can link to the starting screen state, a *When* to the element being used (e.g. Submit button), and a *Then* to the resulting state (e.g. a field's error state). Developers and testers can then click straight from a written criterion to see what it looks like, making a faster connection between what is written and what is designed, and reducing misinterpretation. Link where it adds clarity rather than in every cell. Use Figma's "Copy link to selection" so each link opens the exact element and check to ensure that links still resolve correctly when designs are reorganized.

A7. Acceptance criteria: two complementary formats

Choose based on the return on time invested in defining the acceptance criteria.

Done-When is quick to write and may be enough where the team shares a clear understanding of what needs to be done. Each row has a short "What" label and the "Done-When" condition under which it is accepted as complete; links to the relevant Figma element can sit alongside the condition.

Given-When-Then (Gherkin notation) takes longer, but adds precision where there is complexity, typically when several roles, states or exceptions are involved. A work item can use both: Done-When for what is well understood, and Given-When-Then for when it is necessary to capture precision and complexity. Written as a table, it also becomes a review surface that stakeholders can review before work starts.

A8. Given-When-Then conventions

Name roles (e.g. Content Creator, Approver) rather than "the user". Combining conditions or actions with AND, OR, or other logical operators is fine in any column where it makes sense. Make each Then observable. Cover more than the happy path: validation, permissions, alternate paths, notifications etc.

© 2026 Lionel Lam. Structured Agility: Requirements Definition Template. CC BY 4.0

Licence and attribution

“Structured Agility: Requirements Definition Template” by Lionel Lam is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0).

- Licence: https://creativecommons.org/licenses/by/4.0/
- Companion to: A High-level Guide to Structured Agility

You are free to

- **Share:** copy and redistribute the material in any medium or format, for any purpose, even commercially.
- **Adapt:** remix, transform and build upon the material for any purpose, even commercially.

Under the following terms

- **Attribution:** you must give appropriate credit, provide a link to the licence, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **No additional restrictions:** you may not apply legal terms or technological measures that legally restrict others from doing anything the licence permits.

How to attribute

- **If you use it as is:** “Structured Agility: Requirements Definition Template” by Lionel Lam, licensed under CC BY 4.0 (https://creativecommons.org/licenses/by/4.0/).
- **If you adapt it:** “Adapted from ‘Structured Agility: Requirements Definition Template’ by Lionel Lam, licensed under CC BY 4.0. Changes: [describe your changes].”

Disclaimers

- **Basis of practices:** the methods described draw on industry practices and the author’s own experience. Teams should tailor them to their context and project needs.
- **No warranty:** this template is provided as is, without warranties of any kind. See the licence for full terms.
- **Authorship and AI assistance:** the practices, conventions and rationale in this document reflect the author’s own expertise and experience in delivering projects and products. AI (Anthropic’s Claude) was used as a drafting and editing aid. The author directed the work, reviewed and revised the content, and is responsible for it. The worked example is fictional.